



# 多分支 if-else if 与 switch



# 为什么需要多分支?



现实中的判断往往不止两种结果:

- 成绩等级: 优秀、良好、及格、不及格
- 星期几: 周一、周二、.....、周日
- 运算符: 加、减、乘、除

多分支结构让程序能处理多种可能性, 更加灵活。

# if-else if-else 多分支结构



语法：

```
if (条件1) {  
    // 条件1为真时执行  
} else if (条件2) {  
    // 条件1为假且条件2为真时执行  
} else if (条件3) {  
    // 条件1、2为假且条件3为真时执行  
} else {  
    // 所有条件都为假时执行  
}
```

- 从上到下依次判断，遇到第一个真条件就执行对应分支，然后跳出整个结构
- **else** 是可选的，可以没有

# if-else if 示例：成绩等级



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int score;
    cin >> score;
    if (score >= 90) {
        cout << "优秀";
    } else if (score >= 80) {
        cout << "良好";
    } else if (score >= 60) {
        cout << "及格";
    } else {
        cout << "不及格";
    }
}
```

输入 85 → 输出 良好

输入 59 → 输出 不及格

# 注意：条件顺序很重要



错误写法（可能导致逻辑错误）：

```
if (score >= 60) {  
    cout << "及格";  
} else if (score >= 80) {    // 永远不会执行，因为 >=80 也 >=60  
    cout << "良好";  
}
```

正确做法：将范围小的条件写在前面，或者使用精确范围：

```
if (score >= 90) {  
    // ...  
} else if (score >= 80) {    // 隐含 < 90  
    // ...  
} else if (score >= 60) {    // 隐含 < 80  
    // ...  
}
```

# switch 语句的基本格式



当判断条件是一个整型或字符型的确定值时，`switch` 更简洁。

语法：

```
switch (表达式) {  
    case 值1:  
        // 当表达式 == 值1 时执行  
        break;  
    case 值2:  
        // 当表达式 == 值2 时执行  
        break;  
    // ...  
    default:  
        // 所有 case 都不匹配时执行  
}  

```

- `break` 用于跳出 `switch`，如果漏写，程序会继续执行下一个 `case`（“穿透”）
- `default` 可省略

# switch 示例：星期几



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int day;
    cin >> day;
    switch (day) {
        case 1: cout << "Monday"; break;
        case 2: cout << "Tuesday"; break;
        case 3: cout << "Wednesday"; break;
        case 4: cout << "Thursday"; break;
        case 5: cout << "Friday"; break;
        case 6: cout << "Saturday"; break;
        case 7: cout << "Sunday"; break;
        default: cout << "Invalid day";
    }
    return 0;
}
```

# switch 的穿透现象 (fall-through)



如果某个 `case` 没有 `break`，程序会继续执行下一个 `case` 的代码，直到遇到 `break` 或 `switch` 结束。

```
int x = 2;
switch (x) {
    case 1: cout << "1";
    case 2: cout << "2";    // 执行
    case 3: cout << "3";    // 也会执行! 因为没有 break
}
// 输出 "23"
```

**利用穿透：**有时故意省略 `break` 来处理多个值相同的逻辑：

```
switch (grade) {
    case 'A':
    case 'B': cout << "优秀"; break;
    case 'C': cout << "中等"; break;
    default: cout << "不及格";
}
```

# if-else if 与 switch 的选择



| 场景                              | 推荐结构       | 原因             |
|---------------------------------|------------|----------------|
| 判断区间 (如 $\geq 90$ , $\geq 80$ ) | if-else if | 条件灵活, 支持复杂表达式  |
| 判断离散值 (如 1,2,3)                 | switch     | 结构清晰, 性能略好     |
| 判断字符 (如 'a','b')                | switch     | 支持 char 类型     |
| 涉及逻辑运算符 (&&,   )                | if-else if | switch 不支持复杂逻辑 |



# 逻辑运算符



逻辑运算符用于组合多个条件：

| 运算符 | 含义    | 示例                                         | 结果      |
|-----|-------|--------------------------------------------|---------|
| &&  | 与（并且） | <code>a &gt; 0 &amp;&amp; a &lt; 10</code> | 两个都真才真  |
|     | 或（或者） | <code>a == 0    b == 0</code>              | 至少一个真就真 |
| !   | 非（取反） | <code>!(a &gt; 0)</code>                   | 条件为假时真  |

优先级： `!` > `&&` > `||` （但建议用括号明确优先级）



# 逻辑运算符示例



```
int age = 20;
bool isStudent = true;

// 判断是否青少年（年龄13~19）
if (age >= 13 && age <= 19) {
    cout << "青少年";
}

// 判断是否可以优惠（学生或老人）
if (isStudent || age >= 60) {
    cout << "可以优惠";
}

// 判断不是周末
bool isWeekend = false;
if (!isWeekend) {
    cout << "今天是工作日";
}
```

# 实战①：计算器（四则运算）



## 题目描述

输入两个整数和一个运算符（`+`、`-`、`*`、`/`），输出运算结果。

- 输入：`a op b`，如 `3 + 5`（运算符为字符）
- 输出：运算结果（若除数为0，输出 `Error`）

## 样例

输入：`10 / 3` → 输出：`3`（整数除法）

输入：`5 % 2` → 输出：`1`（取余）

# 计算器 - 参考代码 (if-else if)



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int a, b;
    char op;
    cin >> a >> op >> b;
    if (op == '+') {
        cout << a + b;
    } else if (op == '-') {
        cout << a - b;
    } else if (op == '*') {
        cout << a * b;
    } else if (op == '/') {
        if (b == 0) cout << "Error";
        else cout << a / b;
    } else {
        cout << "Invalid operator";
    }
    return 0;
}
```

# 计算器 - 参考代码 (switch)



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int a, b;
    char op;
    cin >> a >> op >> b;
    switch (op) {
        case '+': cout << a + b; break;
        case '-': cout << a - b; break;
        case '*': cout << a * b; break;
        case '/':
            if (b == 0) cout << "Error";
            else cout << a / b;
            break;
        default: cout << "Invalid operator";
    }
    return 0;
}
```

# 实战②：判断月份的天数



## 题目描述

输入年份和月份，输出该月的天数（考虑闰年）。

- 输入：两个整数，年份 月份
- 输出：该月的天数

**闰年规则：**能被4整除但不能被100整除，或能被400整除。

## 样例

输入：2024 2 → 输出：29

输入：2023 2 → 输出：28

# 判断月份天数 - 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int year, month;
    cin >> year >> month;
    int days;
    switch (month) {
        case 1: case 3: case 5: case 7:
        case 8: case 10: case 12:
            days = 31; break;
        case 4: case 6: case 9: case 11:
            days = 30; break;
        case 2:
            if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)
                days = 29;
            else
                days = 28;
            break;
        default: days = -1; // 非法月份
    }
    if (days != -1) cout << days;
    else cout << "Invalid month";
    return 0;
}
```

# 实战③：字符分类



## 题目描述

输入一个字符，判断它是大写字母、小写字母、数字还是其他字符。

- 大写字母：A ~ Z
- 小写字母：a ~ z
- 数字：0 ~ 9

输入：一个字符

输出：upper / lower / digit / other

## 样例

输入：G → 输出：upper

输入：5 → 输出：digit

# 字符分类 - 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    char c;
    cin >> c;
    if (c >= 'A' && c <= 'Z') {
        cout << "upper";
    } else if (c >= 'a' && c <= 'z') {
        cout << "lower";
    } else if (c >= '0' && c <= '9') {
        cout << "digit";
    } else {
        cout << "other";
    }
    return 0;
}
```

# 实战④：判断三角形的类型



## 题目描述

输入三个整数（代表三条边长），判断它们能否构成三角形，如果能，进一步判断是什么三角形（等边、等腰、普通）。

- 判断条件：任意两边之和大于第三边
- 等边：三边相等
- 等腰：有两边相等（但不是等边）
- 普通：不满足以上

## 样例

输入：3 3 3 → 输出：等边三角形

输入：3 4 5 → 输出：普通三角形

输入：1 1 3 → 输出：不能构成三角形

# 判断三角形类型 - 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int a, b, c;
    cin >> a >> b >> c;
    if (a + b > c && a + c > b && b + c > a) {
        if (a == b && b == c) {
            cout << "等边三角形";
        } else if (a == b || a == c || b == c) {
            cout << "等腰三角形";
        } else {
            cout << "普通三角形";
        }
    } else {
        cout << "不能构成三角形";
    }
    return 0;
}
```

# 注意事项与小技巧



| 要点                | 说明                 |
|-------------------|--------------------|
| if-else if 条件顺序   | 将范围小的条件放在前面，避免逻辑覆盖 |
| switch 中别忘了 break | 否则发生穿透，除非有意利用      |
| 逻辑运算符短路           | && 和 `             |
| 使用括号明确优先级         | 复杂条件用括号分组，提高可读性    |
| switch 只支持整型和字符   | 不能用于浮点数或字符串        |

# 本节回顾



- ✓ **if-else if-else** —— 多分支，处理多个互斥条件
- ✓ **switch 语句** —— 离散值判断，结构清晰
- ✓ **逻辑运算符** —— `&&`（与）、`||`（或）、`!`（非）
- ✓ **实战应用** —— 计算器、月份天数、字符分类、三角形判断

# 下节课预告



## 循环结构 —— while 和 for

- `while` 循环
- `do-while` 循环
- `for` 循环
- 循环控制: `break` 和 `continue`
- 综合练习

“循环让程序可以重复执行，是处理大量数据的基础。”

# 谢谢大家！



## 掌握多分支，让程序能应对复杂世界的多样性！

## Q & A